

FLUTTER & DART

COMPLETE COURSE BOOK

From Beginner to Production-Ready App Developer

Ch 1
Variables & Null Safety

Ch 2
Control Flow

Ch 3
Functions

Ch 4
Collections

Ch 5
OOP & Classes

Ch 6
First Flutter App

Ch 7
Layouts

Ch 8
Lists & Scrolling

Ch 9
Navigation

Ch 10
State Management

Ch 11
REST APIs

Ch 12
Firebase

Ch 13
Architecture

Ch 14
Testing & Deployment

CONTENTS

Table of Contents

Your complete learning map

PART 1 — DART FUNDAMENTALS

Chapter 1 — Variables, Data Types & Null Safety

- 1.1 Variables and types
- 1.2 var, final, const
- 1.3 Null safety
- 1.4 Type conversion
- 1.5 Practice + Solutions

Chapter 2 — Control Flow — if, loops, switch

- 2.1 if/else
- 2.2 Operators
- 2.3 Loops
- 2.4 switch/case
- 2.5 Practice + Solutions

Chapter 3 — Functions & Parameters

- 3.1 Basic functions
- 3.2 Named parameters
- 3.3 Callbacks
- 3.4 Practice + Solutions

Chapter 4 — Collections — List, Map, Set

- 4.1 List
- 4.2 Map
- 4.3 Set
- 4.4 Nested collections
- 4.5 Practice + Solutions

Chapter 5 — OOP — Classes, Inheritance, Mixins

- 5.1 Classes & objects
- 5.2 Constructors
- 5.3 Inheritance
- 5.4 Abstract + Mixins
- 5.5 Practice + Solutions

PART 2 — FLUTTER CORE

Chapter 6 — Your First Flutter App

- 6.1 How Flutter works
- 6.2 Project structure
- 6.3 Widgets
- 6.4 StatelessWidget
- 6.5 StatefulWidget
- 6.6 Practice

Chapter 7 — Layouts — Column, Row, Container, Stack

- 7.1 Container
- 7.2 Column & Row
- 7.3 Expanded
- 7.4 Stack
- 7.5 Helper widgets
- 7.6 Practice

Chapter 8 — Lists, GridView & Scrolling

- 8.1 ListView.builder
- 8.2 GridView
- 8.3 Pull-to-refresh
- 8.4 Practice

Chapter 9 — Navigation — Screens & Routes

- 9.1 Navigator push/pop
- 9.2 Named routes
- 9.3 Bottom navigation
- 9.4 Practice

PART 3 — ADVANCED FLUTTER

Chapter 10 — State Management — Provider & Riverpod

- 10.1 The problem
- 10.2 Provider
- 10.3 Riverpod
- 10.4 Practice

Chapter 11 — REST APIs & Local Storage

- 11.1 HTTP basics
- 11.2 Dio
- 11.3 Models
- 11.4 Local storage
- 11.5 Practice

Chapter 12 — Firebase — Auth, Firestore & Storage

- 12.1 Setup
- 12.2 Authentication
- 12.3 Firestore

- 12.4 Storage
- 12.5 Practice

Chapter 13 — App Architecture & Clean Code

- 13.1 Why architecture
- 13.2 Clean arch layers
- 13.3 Folder structure
- 13.4 Reusable components

Chapter 14 — Testing, Optimization & Deployment

- 14.1 Unit tests
- 14.2 Widget tests
- 14.3 Performance
- 14.4 Play Store publishing

CHAPTER 1

Variables, Data Types & Null Safety

The foundation of every Dart program — understand this and everything else follows

1.1 What is a Variable?

A **variable** is a named container that holds data in your program's memory. Think of it like a labelled box — the label is the variable name, and the box holds the value. Every app you build will use hundreds of variables: the user's name, a product's price, whether someone is logged in, and so on.

In Python you can write `name = "Ali"` and Python silently guesses the type. **Dart is different** — you must declare the type upfront. This is called *static typing*, and it is one of Dart's biggest strengths.

■ Why Dart's Type System Saves You Hours

Python lets you write: `x = 'Ali'` then later: `x = 42` — zero error.
If you then call `x.toUpperCase()` your app crashes at runtime.

Dart PREVENTS this at compile time — before your app even runs.
If `x` is a String, you can never accidentally put an int inside it.

Result: Dart catches entire categories of bugs that Python/JS miss entirely.

1.2 All Basic Data Types

Type	What It Stores	Example	Notes
int	Whole numbers	<code>int age = 25;</code>	No decimal point allowed
double	Decimal numbers	<code>double price = 9.99;</code>	Always has a . decimal
String	Text	<code>String name = "Ali";</code>	Single or double quotes

Type	What It Stores	Example	Notes
bool	True or false	<code>bool ok = true;</code>	Only two possible values
var	Type inferred	<code>var city = "Karachi";</code>	Dart detects type auto
dynamic	Any type	<code>dynamic x = 42;</code>	Avoid — loses safety

Let us look at all of these in action together in one program:

[illegible]

Every single statement in Dart ends with a semicolon ;

Forgetting the semicolon is the single most common beginner mistake.

Dart gives you three keywords for declaring variables. They control whether a variable can change its value later. Choosing the right one is important for both code safety and performance.

Keyword	Can Change?	Determined When?	Real-World Example
var	✓ Yes	Anytime	Search query, counter, score
final	✗ Once	When app runs	User ID from Firebase, API response
const	✗ Never	Before app runs	App name, tax rate, base URL

© Flutter & Dart Complete Course

```
21 print(appName); // TechShop
22 print(taxRate); // 0.18
23
24 // This would be an ERROR – DateTime.now() is not known before runtime:
25 // const currentTime = DateTime.now(); // ERROR!
26 final currentTime = DateTime.now(); // final is correct here
27 }
28
29 String getUserId() => "user_abc123"; // simulates getting ID from server
```

■ Simple Rule to Remember

- const → app name, URLs, tax rate, fixed numbers that never change
- final → data you receive while the app is running (API, Firebase, input)
- var → everything else that you need to change later

In Flutter: almost all widget properties should be final.

1.4 Null Safety — Dart's Most Important Feature

In Python and JavaScript, any variable can be None or null without any warning. This causes the most common crash in software: a **NullPointerException** — trying to use a value that doesn't exist.

Dart completely solves this. You must **explicitly decide** whether a variable can be null by adding a `?` after the type name. Non-nullable variables are *guaranteed* to always have a value — Dart enforces this at compile time, before your app even runs.

[illegible]

[illegible]

X Most Important Rule About !

WRONG: `print(user!.name!.toUpperCase());` // double danger!

```
RIGHT: print(user?.name?.toUpperCase() ?? 'Unknown');
```

The ! operator says 'trust me, this is not null'.

If you are wrong, the app crashes in production.

Only use ! when you are 100% absolutely certain the value is not null.

Beginners who overuse ! completely defeat the purpose of null safety.

Here is a practical real-world example using all four null-handling techniques:

```
1 void main() {
2     // Simulating data that came from a user profile
3     // Some fields were filled in, some were left empty
```


[illegible]

■ Always Use tryParse, Not parse

```
int.parse('abc')    → CRASHES the app with a FormatException
int.tryParse('abc') → returns null safely — NO crash
```

In real apps, data from users or APIs is unpredictable.
Always use `tryParse` when the value might not be a valid number.

1.6 Practice Questions

Try each exercise on **dartpad.dev** — it runs Dart in your browser with zero setup. Attempt each question yourself before reading the solution.

Create a program that defines these variables:

- Calculate the tax amount and final price.
Print this output using string interpolation:

⇒ **Exercise 2 — Medium (20 min)**

You received this user data from an API (simulate it as a Map):

1. Extract each value safely using ?? for defaults
2. age defaults to 0 if null, phone defaults to 'Not provided'
3. Print a formatted user card:

12


```
26 // Code : No discount code
```

[illegible]

```
12 return "Age cannot be negative";
13 }
14 if (age < 18) {
15   return "Must be 18 or older";
16 }
17 // Check email
18 if (email == null) {
19   return "Email is required";
20 }
21 if (!email.contains("@")) {
22   return "Enter a valid email";
23 }
24 // All good!
25 return "Valid user: $name, $age, $email";
26 }
27
28 void main() {
29   print(validateUser(null, 25, "ali@gmail.com")); // Name is required
30   print(validateUser("", 25, "ali@gmail.com")); // Name is required
31   print(validateUser("Ali", null, "ali@gmail.com")); // Age is required
32   print(validateUser("Ali", -5, "ali@gmail.com")); // Age cannot be negative
33   print(validateUser("Ali", 16, "ali@gmail.com")); // Must be 18 or older
34   print(validateUser("Ali", 25, null)); // Email is required
35   print(validateUser("Ali", 25, "invalidemail")); // Enter a valid email
36   print(validateUser("Ali", 25, "ali@gmail.com")); // Valid user: Ali, 25, ali@gmail.com
37 }
```

CHAPTER 2

Control Flow — if, Loops, switch

Make decisions and repeat actions — the logic engine of every app

2.1 if / else if / else — Making Decisions

Control flow is how your program makes decisions. Without it, your code would just run top to bottom every time, doing the same thing regardless of what data it receives. Control flow lets your app respond intelligently.

```
1 void main() {  
2     int score = 75;  
3  
4     // Basic if / else if / else  
5     if (score >= 90) {  
6         print("Grade: A – Excellent!");  
7     } else if (score >= 80) {  
8         print("Grade: B – Very Good");  
9     } else if (score >= 70) {  
10        print("Grade: C – Good");  
11    } else if (score >= 60) {  
12        print("Grade: D – Pass");  
13    } else {  
14        print("Grade: F – Failed");  
15    }  
16    // Output: Grade: C – Good  
17  
18    // ■■ Real App Example: Login Validation ████████████████████████████████  
19    String email = "ali@gmail.com";  
20    String password = "mypassword";  
21    bool verified = true;  
22  
23    if (email.isEmpty) {  
24        print("Error: Email is required");  
25    } else if (password.isEmpty) {  
26        print("Error: Password is required");  
27    } else if (password.length < 6) {  
28        print("Error: Password must be at least 6 characters");
```



```
29 } else if (!verified) {
30   print("Error: Please verify your email first");
31 } else {
32   print("Login successful! Welcome, $email");
33   // Output: Login successful! Welcome, ali@gmail.com
34 }
35 }
```

2.2 Comparison and Logical Operators

Operator	Meaning	Example	Result
==	Equal to	5 == 5	true
!=	Not equal to	5 != 3	true
>	Greater than	10 > 5	true
<	Less than	3 < 7	true
>=	Greater or equal	5 >= 5	true
<=	Less or equal	4 <= 6	true
&&	AND — both must true	true && false	false
	OR — one must true	true false	true
!	NOT — flip the bool	!true	false

```
1 void main() {
2   bool isLoggedIn = true;
3   bool isAdmin = false;
4   int age = 20;
5
6   // && — BOTH conditions must be true
7   if (isLoggedIn && age >= 18) {
8     print("Welcome, adult user!"); // ✓ prints this
9   }
10
11  // || — AT LEAST ONE condition must be true
12  if (isLoggedIn || isAdmin) {
13    print("You have access"); // ✓ prints this (isLoggedIn is true)
14  }
15 }
```

```

16 // ! – FLIP the boolean
17 if (!isAdmin) {
18   print("You are not an admin"); // ✓ prints this
19 }
20
21 // ■■ Ternary operator: short if/else in one line ■■■■■■■■■■■■
22 // condition ? value_if_true : value_if_false
23 String label = isLoggedIn ? "Sign Out" : "Sign In";
24 String status = age >= 18 ? "Adult" : "Minor";
25 int max = age > 25 ? age : 25;
26
27 print(label); // Sign Out
28 print(status); // Adult
29 print(max); // 25
30
31 // In Flutter this looks like:
32 // Text(isLoggedIn ? "Welcome back!" : "Please sign in")
33 // color: isOnline ? Colors.green : Colors.grey
34 }

```

2.3 Loops — Repeating Actions

Loops let you repeat code without copy-pasting it. Every app uses loops — building a list of product cards, processing API data, validating multiple fields.

```

1 void main() {
2
3   // ■■ for loop – when you know how many times to repeat ■■■■■
4   for (int i = 1; i <= 5; i++) {
5     print("Count: $i");
6   }
7   // Count: 1, Count: 2, Count: 3, Count: 4, Count: 5
8
9   // Count down
10  for (int i = 5; i >= 1; i--) {
11    print("Countdown: $i");
12  }
13
14  // Skip by 2 (even numbers)
15  for (int i = 0; i <= 10; i += 2) {
16    print(i); // 0, 2, 4, 6, 8, 10
17  }
18
19  // ■■ for-in – loop through every item in a List ■■■■■■■■■■■■

```

Here is a real-world example — calculating a shopping cart total:

```
1 void main() {
2     // Real app: loop through cart items and calculate total
3     List<Map<String, dynamic>> cartItems = [
4         {"name": "iPhone Case", "price": 15.99, "qty": 2},
5         {"name": "USB Cable", "price": 9.99, "qty": 3},
6         {"name": "Screen Guard", "price": 7.99, "qty": 1},
7         {"name": "Power Bank", "price": 49.99, "qty": 1},
8     ];
9
10    double total = 0;
11    int items = 0;
```

2.4 switch / case

Use **switch** when you are checking one variable against many fixed values. It is cleaner than a long chain of if/else if. Always include a break after each case and a default for unexpected values.

```
1 void main() {
2
3 // ■■ Basic switch
4 String day = "Friday";
5
6 switch (day) {
7 case "Monday":
8     print("Start of work week – tough day!");
9     break;
```


✗ Common Mistake — Missing break

```
switch (color) {  
  case "red":  
    print("Red");  
    // NO BREAK – falls through to blue!  
  case "blue":  
    print("Blue"); // Also prints even if color is red!  
    break;  
}
```

Always add break; after each case unless you intentionally want fall-through.

2.5 Practice Questions

⇒ Exercise 1 — Easy (10 min)

Write a for loop that prints the 7 times table:

```
7 × 1 = 7  
7 × 2 = 14  
7 × 3 = 21  
... (up to 7 × 10 = 70)
```

⇒ Exercise 2 — Medium (20 min)

You have this price list:

```
List<double> prices = [9.99, 49.99, 150.00, 19.99, 200.00, 5.00, 75.00]
```

1. Loop through all prices
2. Print: 'Cheap' if < 20, 'Mid-range' if 20-100, 'Premium' if > 100
3. Count items in each category
4. Calculate and print the total
5. Print a summary at the end showing counts and total

⇒ Exercise 3 — Hard (30 min)

Build a number guessing game using a while loop.

The secret number is 42.

Simulate guesses with a List: [10, 80, 35, 55, 42]

Rules:

- For each guess print: 'Too low!', 'Too high!', or 'Correct!'
- Track how many attempts it took
- Stop the loop as soon as the number is found
- After finding it, print: 'Found in X attempts!'
- If all guesses are exhausted: 'Out of guesses! Number was 42'

Solutions


```
17 print(" CORRECT! ■ You found $secret!");
18 found = true;
19 break; // stop looping – we found it
20 }
21 }
22
23 // After the loop
24 if (found) {
25   print("Found in $attempts attempt${attempts == 1 ? '' : 's'}!");
26 } else {
27   print("Out of guesses! The number was $secret");
28 }
29 }
30 // Guess 1: 10
31 // Too low! Go higher.
32 // Guess 2: 80
33 // Too high! Go lower.
34 // Guess 3: 35
35 // Too low! Go higher.
36 // Guess 4: 55
37 // Too high! Go lower.
38 // Guess 5: 42
39 // CORRECT! ■ You found 42!
40 // Found in 5 attempts!
```


■ Why This Section is Critical

Every single Flutter widget uses named parameters.

```
ElevatedButton(onPressed: myFunc, child: Text('Click'))
Container(width: 100, height: 50, color: Colors.red)
Text('Hello', style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold))
```

If you do not understand named parameters, you cannot use Flutter.

```

1 void main() {
2
3 // ■■ Positional parameters – must be in exact order ■■■■■■■■
4 void showProduct(String name, double price, String category) {
5     print("$name | \$$price | $category");
6 }
7 showProduct("iPhone", 999.99, "Phones"); // must be in this order
8 // showProduct(999.99, "iPhone", "Phones"); // ERROR – wrong order
9
10 // ■■ Named parameters – use {} curly braces ■■■■■■■■■■■■■■■■■■■■
11 // Call them in ANY order using the parameter name
12 void showProductNamed({
13     required String name, // required – caller MUST provide this
14     required double price, // required – caller MUST provide this
15     String category = "General", // optional – has a default value
16     bool inStock = true, // optional – has a default value
17 }) {
18     String status = inStock ? "■ In Stock" : "■ Out of Stock";
19     print("$name | \${price.toStringAsFixed(2)} | $category | $status");
20 }
21
22 // Named – order does NOT matter! Very flexible.
23 showProductNamed(name: "iPhone 15", price: 999.99);
24 // iPhone 15 | $999.99 | General | ■ In Stock
25
26 showProductNamed(price: 849.99, name: "Samsung S24", category: "Phones");
27 // Samsung S24 | $849.99 | Phones | ■ In Stock
28
29 showProductNamed(name: "iPad Pro", price: 1099.0, inStock: false);
30 // iPad Pro | $1099.00 | General | ■ Out of Stock
31
32 // showProductNamed(price: 99.99); // ERROR – name is required!
33 }

```

■ required vs Optional Named Parameters

required → the caller **MUST** provide this argument

if they forget, Dart gives a compile-time error

No required keyword + default value → the parameter is optional

the default is used if not provided

This is exactly how Flutter widgets work:

Text(text) ← required (no default makes sense)

Text(softWrap: true) ← optional with sensible default

```

1 // ■■■ Practical example: building a user profile function ■■■■■■■■
2 void createUserProfile({
3     required String name,
4     required String email,
5     int age = 0,
6     String? phone, // nullable optional – defaults to null
7     String plan = "free",
8     bool verified = false,
9 }) {
10    print("■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■");
11    print(" Name : $name");
12    print(" Email : $email");
13    print(" Age : ${age > 0 ? age.toString() : 'Not provided'}");
14    print(" Phone : ${phone ?? 'Not provided'}");
15    print(" Plan : ${plan.toUpperCase()}");
16    print(" Verified : ${verified ? 'Yes ■' : 'No ■'}");
17    print("■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■");
18 }
19
20 void main() {
21     // Only required fields:
22     createUserProfile(name: "Ali Hassan", email: "ali@gmail.com");
23
24     // With some optional fields:
25     createUserProfile(
26         name: "Sara Ahmed",
27         email: "sara@gmail.com",
28         age: 25,
29         plan: "premium",
30         verified: true,
31     );
32 }

```



```
38 print(passing); // [85, 92, 95, 88]
39
40 // any: true if AT LEAST ONE item matches
41 bool anyFailed = scores.any((s) => s < 60);
42 print(anyFailed); // false
43
44 // every: true if ALL items match
45 bool allPassed = scores.every((s) => s >= 60);
46 print(allPassed); // true
47
48 // reduce: combine all items into a single value
49 int total = scores.reduce((sum, s) => sum + s);
50 print(total); // 498
51 double avg = total / scores.length;
52 print(avg); // 83.0
53
54 // forEach: do something with each item
55 names.forEach((name) => print(" • $name"));
56
57 // In Flutter, you will write code like this constantly:
58 // children: products.map((p) => ProductCard(product: p)).toList()
59 // items.where((item) => item.isActive).toList()
60 }
```

3.4 Practice Questions

⇒ Exercise 1 — Easy (10 min)

Write these 3 functions and call them to verify output:

1. `celsiusToFahrenheit(double c) → (c × 9/5) + 32`

`celsiusToFahrenheit(0) = 32.0`

`celsiusToFahrenheit(100) = 212.0`

2. `describeTemperature(double celsius) → String`

Below 0 → 'Freezing ❄'

0 to 15 → 'Cold ■'

16 to 30 → 'Warm ■'

Above 30 → 'Hot ■'

3. Loop through `[0, -10, 25, 37, 100]` and print both for each

⇒ Exercise 2 — Medium (25 min)

Write a function `printReceipt` that accepts:

- storeName (String, optional, default 'MyStore')
- items (List<Map<String,dynamic>>, required)
each map has: name, price, qty
- discount (double, optional, default 0.0) — percentage 0.0-1.0

It should print a formatted receipt showing:

- Store name in header
- Each item: name, qty, unit price, line total
- Subtotal, discount amount, final total

Test with at least 4 items and a 15% discount

⇒ **Exercise 3 — Hard (35 min)**

Write a higher-order function that takes a List and a function, applies the function to every item, and returns the new list.

Signature: `List<T> transform<T>(List<T> items, T Function(T) fn)`

Then write these transformation functions:

1. `applyTax(double price)` — adds 18% tax
2. `applyDiscount(double price)` — removes 20%
3. `roundToNearest5(double n)` — rounds to nearest 5
4. `formatPrice(double n)` — returns 'PKR X,XXX.XX' formatted string

Apply each transformation to [100.0, 250.0, 399.0, 750.0, 1200.0]
and print all results in a table format

Solutions

```
1 // ■■■ SOLUTION 1
2 double celsiusToFahrenheit(double c) => (c * 9 / 5) + 32;
3
4 String describeTemperature(double c) {
5     if (c < 0) return "Freezing ❄️";
6     if (c <= 15) return "Cold ❄️";
7     if (c <= 30) return "Warm 🌡️";
8     return "Hot 🔥";
9 }
```



```

9      }
10
11     void main() {
12         List<double> temps = [0, -10, 25, 37, 100];
13
14         for (double c in temps) {
15             double f = celsiusToFahrenheit(c);
16             String desc = describeTemperature(c);
17             print("${c}°C = ${f.toStringAsFixed(1)}°F → $desc");
18         }
19     }
20
21     // 0.0°C = 32.0°F → Freezing *
22     // -10.0°C = 14.0°F → Freezing *
23     // 25.0°C = 77.0°F → Warm ■
24     // 37.0°C = 98.6°F → Hot ■
25     // 100.0°C = 212.0°F → Hot ■

```

```

1 // ■■■ SOLUTION 2
2
3 void printReceipt({
4     String storeName = "MyStore",
5     required List<Map<String, dynamic>> items,
6     double discount = 0.0,
7 }) {
8     double subtotal = 0;
9
10    print("■${" * 38}■");
11    print("■ ${storeName.padRight(36)}■");
12    print("■${" * 38}■");
13
14    for (var item in items) {
15        double lineTotal = (item["price"] as double) * (item["qty"] as int);
16        subtotal += lineTotal;
17        String left = "${item['name']} x${item['qty']}";
18        String right = "\${lineTotal.toStringAsFixed(2)}";
19        print("■ ${left.padRight(26)} ${right.padLeft(9)} ■");
20    }
21
22    double discAmount = subtotal * discount;
23    double total = subtotal - discAmount;
24
25    print("■${" * 38}■");
26    print("■ Subtotal ${(" + "\${subtotal.toStringAsFixed(2)}").padLeft(28)} ■");
27    if (discount > 0) {

```

```

27 print("■ Discount ${ (discount*100).toInt()}%
    ${("-\${discAmount.toStringAsFixed(2)}").padLeft(24)} ■");
28 }
29 print("■ TOTAL ${("\${total.toStringAsFixed(2)}").padLeft(30)} ■");
30 print("■${"■" * 38}■");
31 }
32
33 void main() {
34     printReceipt(
35         storeName: "TechMart PK",
36         discount: 0.15,
37         items: [
38             {"name": "iPhone Case", "price": 15.99, "qty": 2},
39             {"name": "USB Cable", "price": 9.99, "qty": 3},
40             {"name": "Screen Guard", "price": 7.99, "qty": 1},
41             {"name": "Power Bank", "price": 49.99, "qty": 1},
42         ],
43     );
44 }

```

```

1 // ■■■ SOLUTION 3
2
3 // Generic transform function
4 List<T> transform<T>(List<T> items, T Function(T) fn) =>
5 items.map(fn).toList();
6
7 // Transformation functions
8 double applyTax(double p) => p * 1.18;
9 double applyDiscount(double p) => p * 0.80;
10 double roundToNearest5(double n) => (n / 5).round() * 5.0;
11 String formatPrice(double n) => "PKR ${n.toStringAsFixed(2)}";
12
13 void main() {
14     List<double> prices = [100.0, 250.0, 399.0, 750.0, 1200.0];
15
16     var taxed = transform(prices, applyTax);
17     var discounted = transform(prices, applyDiscount);
18     var rounded = transform(prices, roundToNearest5);
19     var formatted = transform(prices, formatPrice);
20
21     print("Original : $prices");
22     print("After 18% : ${taxed.map((p)=>p.toStringAsFixed(2)).toList()}");
23     print("After -20% : ${discounted.map((p)=>p.toStringAsFixed(2)).toList()}");
24     print("Rounded/5 : $rounded");
25     print("Formatted : $formatted");
26 }

```

```
25 }  
26 // Original : [100.0, 250.0, 399.0, 750.0, 1200.0]  
27 // After 18% : [118.00, 295.00, 470.82, 885.00, 1416.00]  
28 // After -20% : [80.00, 200.00, 319.20, 600.00, 960.00]  
29 // Rounded/5 : [100.0, 250.0, 400.0, 750.0, 1200.0]  
30 // Formatted : [PKR 100.00, PKR 250.00, PKR 399.00, PKR 750.00, PKR 1200.00]
```

CHAPTER 4

Collections — List, Map, Set

Store and work with groups of data — used in every real app

4.1 List — Ordered Collection

A **List** is an ordered sequence of items accessed by index number (starting at 0). It is the most frequently used collection in Flutter — every product list, message list, and widget list uses it.

[illegible]


```

20 // ■■ Adding and updating
21 user["city"] = "Karachi"; // add new key
22 user["age"] = 26; // update existing key
23 user.addAll({"country": "Pakistan", "joined": "2024"});
24 user.putIfAbsent("role", () => "member"); // add only if missing
25
26 // ■■ Removing
27 user.remove("joined");
28
29 // ■■ Checking
30 print(user.containsKey("email")); // true
31 print(user.containsValue("Karachi")); // true
32 print(user.length); // 7
33
34 // ■■ Looping through key-value pairs
35 user.forEach((key, value) {
36   print("$key → $value");
37 });
38
39 // ■■ Get all keys or all values
40 print(user.keys.toList());
41 print(user.values.toList());
42 }

```

4.4 Nested Collections — The Real App Pattern

Real apps receive data from APIs as a **List of Maps** — a list where each item is a map describing one object. This is the most common data structure in Flutter development.

[illegible]

39 }

4.5 Practice Questions

⇒ Exercise 1 — Easy

Given: List<int> scores = [85,92,78,95,60,88,72,91,55,80]

1. Find the highest score using reduce
2. Find the lowest score using reduce
3. Calculate the average
4. Print all scores above 80 (use where)

⇒ Exercise 2 — Medium

Given a List of student maps with 'name' and 'marks' (List<int>):

Write a function `getTopStudents(students, n, minAvg)`

that returns the top N students with avg $\geq$ minAvg, sorted by avg descending.

Include the calculated average in each returned map.

⇒ Exercise 3 — Hard

Build a complete inventory report system:

- Products list with: name, category, price, qty
- Group products by category
- For each category: count, total value (price × qty)
- Find most expensive in each category
- Flag items where qty < 5 as 'LOW STOCK'
- Print a full formatted report

Solutions

```
1 // ■■■ SOLUTION 1
2
3 void main() {
4
5     List<int> scores = [85,92,78,95,60,88,72,91,55,80];
6
7     int highest = scores.reduce((a, b) => a > b ? a : b); // 95
8     int lowest = scores.reduce((a, b) => a < b ? a : b); // 55
9     double avg = scores.reduce((a,b) => a+b) / scores.length; // 79.6
10    List<int> above80 = scores.where((s) => s > 80).toList();
11}
```



```

9
10 print("Highest : $highest"); // 95
11 print("Lowest : $lowest"); // 55
12 print("Average : ${avg.toStringAsFixed(1)}"); // 79.6
13 print("Above 80: $above80"); // [85, 92, 95, 88, 91]
14 }
15
16 // ■■■ SOLUTION 3
17 void main() {
18   List<Map<String,dynamic>> products = [
19     {"name":"iPhone 15", "category":"Phones", "price":999.99, "qty":8},
20     {"name":"Samsung", "category":"Phones", "price":849.99, "qty":3},
21     {"name":"iPad Pro", "category":"Tablets", "price":1099.0, "qty":12},
22     {"name":"MacBook", "category":"Laptops", "price":1299.0, "qty":4},
23     {"name":"AirPods", "category":"Audio", "price":249.99, "qty":20},
24     {"name":"Headphones", "category":"Audio", "price":79.99, "qty":2},
25   ];
26
27   // Group by category
28   Map<String, List<Map<String,dynamic>>> grouped = {};
29   for (var p in products) {
30     String cat = p["category"];
31     grouped.putIfAbsent(cat, () => []);
32     grouped[cat]!.add(p);
33   }
34
35   print("■" * 50);
36   print("INVENTORY REPORT");
37   print("■" * 50);
38
39   grouped.forEach((cat, items) {
40     int count = items.length;
41     double value = items.fold(0, (s,p)=> s + p["price"]*p["qty"]);
42     var mostExp = items.reduce((a,b) => a["price"]>b["price"]?a:b);
43
44     print("
45 [$cat] $count items | Value: \${value.toStringAsFixed(0)}");
46     for (var p in items) {
47       String stock = (p["qty"] as int) < 5 ? " ■ LOW STOCK" : "";
48       print(" • ${p['name'].padRight(12)} qty:${p['qty']} \${p['price']}$stock");
49     }
50     print(" Most expensive: ${mostExp['name']}");
51   });
52   print("

```

```
53 " + "■" * 50);  
54 }
```

CHAPTER 5

OOP — Classes, Objects, Inheritance

Every Flutter widget is a class — master this and Flutter makes complete sense

5.1 What is a Class?

A **class** is a blueprint. From one blueprint you can create many objects. Think of a class like a cookie cutter — the cutter is the class, each cookie is an object. In Flutter, *every widget is a class* — Text, Button, Column, Row — all of them.

```
1 class Car {
2   // Properties (data the object holds)
3   String brand;
4   String color;
5   int year;
6   double price;
7
8   // Constructor – runs when you create an object from this class
9   Car({
10    required this.brand,
11    required this.color,
12    required this.year,
13    required this.price,
14  });
15
16   // Methods (actions the object can perform)
17   void showInfo() {
18     print("$year $brand ($color) – \${price.toStringAsFixed(0)}");
19   }
20
21   bool isExpensive() => price > 50000;
22
23   String get priceTag {
24     if (price < 20000) return "Budget";
25     if (price < 60000) return "Mid-range";
26     return "Luxury";
27   }
28 }
```

```
29
30 void main() {
31   // Create objects FROM the class (like making cookies from a cutter)
32   Car car1 = Car(brand:"Toyota", color:"White", year:2023, price:35000);
33   Car car2 = Car(brand:"BMW", color:"Black", year:2024, price:85000);
34   Car car3 = Car(brand:"Suzuki", color:"Red", year:2022, price:12000);
35
36   car1.showInfo(); // 2023 Toyota (White) – $35000
37   car2.showInfo(); // 2024 BMW (Black) – $85000
38
39   print(car1.isExpensive()); // false
40   print(car2.isExpensive()); // true
41
42   print(car1.priceTag); // Mid-range
43   print(car2.priceTag); // Luxury
44   print(car3.priceTag); // Budget
45
46   // Change a property
47   car1.color = "Silver";
48   car1.showInfo(); // 2023 Toyota (Silver) – $35000
49 }
```

5.2 Constructors — 4 Types

```
1 // ■■ Type 1: Named parameters constructor (most common in Flutter) ■■
2 class Product {
3   String name;
4   double price;
5   String category;
6   bool inStock;
7
8   Product({
9     required this.name,
10    required this.price,
11    this.category = "General",
12    this.inStock = true,
13  });
14
15 // ■■ Type 2: Named constructor – create from JSON ■■■■■■■■■■
16 // Use this when converting API response to a Dart object
17 Product.fromJson(Map<String, dynamic> json)
18   : name = json["name"],
19     price = (json["price"] as num).toDouble(),
20     category = json["category"] ?? "General",
```

```
21 inStock = json["inStock"] ?? true;
22
23 // ■■ Type 3: Named constructor for special cases ■■■■■■■■■■■■
24 Product.outOfStock({required String name, required double price})
25 : name = name,
26 price = price,
27 category = "General",
28 inStock = false;
29
30 // Convert back to JSON (for sending to API)
31 Map<String, dynamic> toJson() => {
32   "name" : name,
33   "price" : price,
34   "category": category,
35   "inStock" : inStock,
36 };
37
38 void display() {
39   String s = inStock ? "■ In Stock" : "■ Out of Stock";
40   print("$name | \${price.toStringAsFixed(2)} | $category | $s");
41 }
42 }
43
44 void main() {
45   // Regular constructor
46   Product p1 = Product(name:"iPhone 15", price:999.99, category:"Phones");
47
48   // From JSON (like you would get from an API)
49   Product p2 = Product.fromJson({
50     "name": "Samsung S24", "price": 849.99, "category": "Phones", "inStock": true
51   });
52
53   // Out of stock constructor
54   Product p3 = Product.outOfStock(name:"iPad Pro", price:1099.0);
55
56   p1.display(); // iPhone 15 | $999.99 | Phones | ■ In Stock
57   p2.display(); // Samsung S24 | $849.99 | Phones | ■ In Stock
58   p3.display(); // iPad Pro | $1099.00 | General | ■ Out of Stock
59
60   print(p1.toJson()); // {name: iPhone 15, price: 999.99, ...}
61 }
```

5.3 Inheritance — Parent and Child Classes

Inheritance lets one class **get all the properties and methods** of another class, then add its own on top. The child class *inherits* from the parent. Use `extends` to inherit.

```
1 // Parent class
2 class Animal {
3   String name;
4   int age;
5
6   Animal({required this.name, required this.age});
7
8   void breathe() => print("$name is breathing");
9   void sleep() => print("$name is sleeping");
10  void describe() => print("Animal: $name (age $age)");
11 }
12
13 // Dog inherits EVERYTHING from Animal, plus adds its own stuff
14 class Dog extends Animal {
15   String breed;
16
17   Dog({required String name, required int age, required this.breed})
18   : super(name: name, age: age); // super() calls Animal's constructor
19
20   // Dog's own method – Animal does not have this
21   void bark() => print("$name says: Woof! 🐶");
22   void fetch() => print("$name fetches the ball!");
23
24   // Override – change what the parent's method does
25   @override
26   void describe() {
27     super.describe(); // first run Animal's version
28     print("Breed: $breed"); // then add extra info
29   }
30 }
31
32 class Cat extends Animal {
33   bool isIndoor;
34
35   Cat({required String name, required int age, required this.isIndoor})
36   : super(name: name, age: age);
37
38   void meow() => print("$name says: Meow! 🐱");
39
40   @override
41   void describe() {
42     super.describe();
```

```
43 print("Type: ${isIndoor ? 'Indoor' : 'Outdoor'} cat");
44 }
45 }
46
47 void main() {
48   Dog dog = Dog(name:"Max", age:3, breed:"Labrador");
49   Cat cat = Cat(name:"Bella", age:2, isIndoor:true);
50
51   // Dog has Animal methods AND its own
52   dog.breathe(); // Max is breathing ← from Animal
53   dog.sleep(); // Max is sleeping ← from Animal
54   dog.bark(); // Max says: Woof! ← Dog's own
55   dog.describe(); // Animal: Max (age 3) ← both layers
56   // Breed: Labrador
57
58   cat.breathe(); // Bella is breathing ← from Animal
59   cat.meow(); // Bella says: Meow! ← Cat's own
60   cat.describe(); // Animal: Bella (age 2)
61   // Type: Indoor cat
62
63   // Polymorphism – treat all animals as Animal type
64   List<Animal> animals = [dog, cat];
65   for (Animal a in animals) {
66     a.breathe(); // works for all of them
67     a.describe(); // each one calls its OWN describe()
68   }
69 }
```

5.4 Abstract Classes

An **abstract class** is a contract — it defines what methods a class **MUST** have, without saying how they work. You cannot create an object from an abstract class. It forces all child classes to implement the required methods.

```
1 abstract class Shape {
2   String color;
3   Shape(this.color);
4
5   // Abstract methods – child MUST implement these
6   double area();
7   double perimeter();
8
9   // Regular shared method – child gets this for free
10  void describe() {
```

```
11 print("${runtimeType} [$color]");
12 print(" Area : ${area().toStringAsFixed(2)}");
13 print(" Perimeter : ${perimeter().toStringAsFixed(2)}");
14 }
15 }
16
17 class Circle extends Shape {
18   double radius;
19   Circle({required this.radius, required String color}) : super(color);
20
21   @override double area() => 3.14159 * radius * radius;
22   @override double perimeter() => 2 * 3.14159 * radius;
23 }
24
25 class Rectangle extends Shape {
26   double width, height;
27   Rectangle({required this.width, required this.height, required String color})
28     : super(color);
29
30   @override double area() => width * height;
31   @override double perimeter() => 2 * (width + height);
32 }
33
34 void main() {
35   // Shape s = Shape("Red"); // ERROR – can't create abstract class!
36
37   List<Shape> shapes = [
38     Circle(radius: 7, color: "Red"),
39     Rectangle(width: 10, height: 5, color: "Blue"),
40     Circle(radius: 3, color: "Green"),
41   ];
42
43   for (var s in shapes) s.describe();
44
45   // Find the largest shape
46   Shape largest = shapes.reduce((a, b) => a.area() > b.area() ? a : b);
47   print("Largest: ${largest.runtimeType}");
48 }
```

5.5 Practice Questions

Build a complete Employee Management System:

Properties: id, name, baseSalary
 Abstract methods: bonus() → double, role() → String
 Regular method: totalSalary() = baseSalary + bonus()
 Regular method: printPayslip() — formatted output

```
Extra: teamSize (int)
bonus() = baseSalary × 0.20 + teamSize × 500
role() = 'Manager'
```

```
Extra: language (String), overtimeHours (double)
bonus() = overtimeHours × 50
role() = 'Developer (Python/Flutter/etc)'
```

Create 4 employees, print all payslips, then print total payroll

Solution

```

1 abstract class Employee {
2     String id, name;
3     double baseSalary;
4
5     Employee({required this.id, required this.name, required this.baseSalary});
6
7     double bonus();
8     String role();
9
10    double get totalSalary => baseSalary + bonus();
11
12    void printPayslip() {
13        print("■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■");
14        print("■ ID : $id");
15        print("■ Name : $name");
16        print("■ Role : ${role()}");
17        print("■ Base : \\\${baseSalary.toStringAsFixed(0)}");
18        print("■ Bonus : \\\${bonus().toStringAsFixed(0)}");
19        print("■ TOTAL : \\\${totalSalary.toStringAsFixed(0)}");

```

```

20 print("■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■");
21 }
22 }
23
24 class Manager extends Employee {
25     int teamSize;
26     Manager({required String id, required String name,
27         required double baseSalary, required this.teamSize})
28         : super(id:id, name:name, baseSalary:baseSalary);
29
30     @override double bonus() => baseSalary * 0.20 + teamSize * 500;
31     @override String role() => "Manager";
32 }
33
34 class Developer extends Employee {
35     String language;
36     double overtimeHours;
37     Developer({required String id, required String name,
38         required double baseSalary, required this.language,
39         this.overtimeHours = 0})
40         : super(id:id, name:name, baseSalary:baseSalary);
41
42     @override double bonus() => overtimeHours * 50;
43     @override String role() => "Developer ($language)";
44 }
45
46 void main() {
47     List<Employee> team = [
48         Manager(id:"M01", name:"Ali Khan", baseSalary:5000, teamSize:8),
49         Manager(id:"M02", name:"Sara Malik", baseSalary:4500, teamSize:5),
50         Developer(id:"D01",name:"Hassan", baseSalary:4000, language:"Flutter",
            overtimeHours:12),
51         Developer(id:"D02",name:"Zara", baseSalary:3800, language:"Python", overtimeHours:6),
52     ];
53
54     for (var e in team) e.printPayslip();
55
56     double grand = team.fold(0, (s, e) => s + e.totalSalary);
57     print("
58 TOTAL PAYROLL: \${grand.toStringAsFixed(0)}");
59 }

```

CHAPTER 6

Your First Flutter App

How Flutter works, the widget tree, and your first real interactive screen

6.1 How Flutter Works

■ The Big Picture

You write Dart code



Flutter Framework (widgets, layout, animations, gestures)



Flutter Engine (C++ — rendering, input, platform calls)



Skia / Impeller (draws every single pixel itself)



Android APK / iOS IPA / Web / Desktop

Flutter does NOT use native Android buttons or iOS text fields.

It paints EVERY pixel on the screen itself.

This is why Flutter looks identical on Android and iOS, and why it is fast.

6.2 Widgets — Everything is a Widget

In Flutter, *everything* you see on screen is a widget. A button? Widget. A text label? Widget. The padding around a button? Widget. The entire screen? Widget. Even the app itself is a widget.

```
1 import 'package:flutter/material.dart';
2
3 void main() {
4   runApp(const MyApp()); // start the Flutter app
5 }
6
7 // The root of your entire app – always StatelessWidget
8 class MyApp extends StatelessWidget {
```

```

9  const MyApp({super.key});
10
11  @override
12  Widget build(BuildContext context) {
13    return MaterialApp(
14      title: 'My First App',
15      debugShowCheckedModeBanner: false, // removes red DEBUG banner
16      theme: ThemeData(
17        colorScheme: ColorScheme.fromSeed(seedColor: Color(0xFF5C4FE8)),
18        useMaterial3: true,
19      ),
20      home: const HomeScreen(), // first screen the user sees
21    );
22  }
23 }
24
25 // Widget tree:
26 // MyApp
27 //   ███ MaterialApp
28 //     ███ HomeScreen (Scaffold)
29 //       ███ AppBar
30 //         █ ███ Text("My App")
31 //           ███ Body
32 //             ███ Center
33 //               ███ Text("Hello Flutter!")

```

6.3 StatelessWidget vs StatefulWidget

Type	When to Use	Example
StatelessWidget	UI never changes — static content	About page, Logo, Profile header
StatefulWidget	UI updates when data changes	Counter, Form, Toggle, Shopping cart

[illegible]

[illegible]

```
54   foregroundColor: Colors.white,
55   ),
56   body: Center(
57     child: Column(
58       mainAxisAlignment: MainAxisAlignment.center,
59       children: [
60         const Text("Count", style: TextStyle(fontSize:16, color:Colors.grey)),
61         Text(
62           "$_count",
63           style: const TextStyle(fontSize:80, fontWeight:FontWeight.bold,
64             color:Color(0xFF5C4FE8)),
65         ),
66         const SizedBox(height:40),
67         Row(
68           mainAxisAlignment: MainAxisAlignment.center,
69           children: [
70             FloatingActionButton(
71               onPressed: _decrement,
72               heroTag: "minus",
73               child: const Icon(Icons.remove),
74             ),
75             const SizedBox(width: 40),
76             FloatingActionButton(
77               onPressed: _increment,
78               heroTag: "plus",
79               backgroundColor: const Color(0xFF5C4FE8),
80               child: const Icon(Icons.add, color:Colors.white),
81             ),
82           ],
83         ),
84       ],
85     ),
86   ),
87 );
88 }
89 }
```

X The 3 Most Common Beginner Flutter Mistakes

1. Forgetting const

`Text('Hello')` – rebuilds every time (slow)
`const Text('Hello')` – Flutter reuses it (fast) ✓

2. Changing state without setState

`_count++;` – screen does NOT update
`setState(() { _count++; });` – screen updates ✓

3. Column overflow — content taller than screen

Wrap Column in `SingleChildScrollView` to fix yellow/black stripes

CHAPTER 7

Layouts — Column, Row, Container, Stack

Build any screen you can imagine — master the Flutter layout system

7.1 Container — The Styled Box

```
1 // Container is a box you can style in every way possible
2 Container(
3   width: double.infinity, // fill parent width completely
4   height: 200,
5
6   // margin = space OUTSIDE (between this and other widgets)
7   // padding = space INSIDE (between border and child content)
8   margin: const EdgeInsets.all(16),
9   padding: const EdgeInsets.all(20),
10
11  decoration: BoxDecoration(
12    // Option A: solid color
13    color: const Color(0xFF5C4FE8),
14
15    // Option B: gradient (cannot use both color AND gradient)
16    // gradient: LinearGradient(colors:[Colors.blue, Colors.purple]),
17
18    // Rounded corners
19    borderRadius: BorderRadius.circular(16),
20
21    // Border
22    border: Border.all(color: Colors.grey.shade300, width: 1),
23
24    // Drop shadow
25    boxShadow: [
26      BoxShadow(
27        color: Colors.black.withOpacity(0.15),
28        blurRadius: 12,
29        offset: const Offset(0, 6), // x=0, y=6 (shadow below)
30      ),
31    ],
32  ),
```


[illegible]

7.2 Column and Row

```

1 // Column stacks children VERTICALLY (top to bottom)
2 Column(
3   mainAxisAlignment: MainAxisAlignment.center,
4   // .start → pack at top (default)
5   // .center → center vertically
6   // .end → pack at bottom
7   // .spaceBetween → space between items, no space at edges
8   // .spaceEvenly → equal space everywhere including edges
9   // .spaceAround → space around each item
10
11   crossAxisAlignment: CrossAxisAlignment.stretch,
12   // .start → align left
13   // .center → center horizontally (default)
14   // .end → align right
15   // .stretch → stretch children to full available width
16
17   children: [
18     Text("Item 1"),
19     SizedBox(height: 12), // gap – use SizedBox for spacing
20     Text("Item 2"),
21     Spacer(), // flexible space – pushes next items down
22     Text("This is at bottom"),
23   ],
24 )
25
26 // Row arranges children HORIZONTALLY (left to right)
27 // mainAxisAlignment controls LEFT-RIGHT positioning
28 // crossAxisAlignment controls UP-DOWN positioning
29 Row(
30   mainAxisAlignment: MainAxisAlignment.spaceBetween,
31   crossAxisAlignment: CrossAxisAlignment.center,
32   children: [

```

```
33 // Icon thumbnail
34 Container(width:70, height:70, color:Colors.grey.shade200,
35 child: const Icon(Icons.phone_iphone, size:40)),
36 // Product info – Expanded takes all remaining width
37 const Expanded(
38 child: Padding(
39 padding: EdgeInsets.symmetric(horizontal:12),
40 child: Column(
41 crossAxisAlignment: CrossAxisAlignment.start,
42 children: [
43 Text("iPhone 15", style:TextStyle(fontWeight:FontWeight.bold)),
44 Text("256GB", style:TextStyle(color:Colors.grey)),
45 ],
46 ),
47 ),
48 ),
49 // Price
50 const Text("\$999", style:TextStyle(fontSize:18, fontWeight:FontWeight.bold)),
51 ],
52 )
```

7.3 Expanded — Sharing Space

```
1 // Expanded fills all remaining space in a Row or Column
2 Row(
3 children: [
4 // Fixed 80px box
5 Container(width:80, height:50, color:Colors.red),
6 // This takes ALL remaining horizontal space
7 Expanded(
8 child: Container(height:50, color:Colors.blue,
9 child: const Center(child: Text("Takes rest"))),
10 ),
11 ],
12 )
13
14 // Multiple Expanded with flex – custom split ratios
15 Row(
16 children: [
17 Expanded(flex:1, child: Container(height:50, color:Colors.red)), // 25%
18 SizedBox(width:8),
19 Expanded(flex:2, child: Container(height:50, color:Colors.green)), // 50%
20 SizedBox(width:8),
21 Expanded(flex:1, child: Container(height:50, color:Colors.blue)), // 25%
```

```
22 ],
23 )
24 // Red=25% Green=50% Blue=25% of total row width
```

7.4 Stack — Overlapping Widgets

```
1 // Stack lets widgets sit on top of each other (like Photoshop layers)
2 Stack(
3   clipBehavior: Clip.none, // allows children to go outside stack bounds
4   children: [
5
6     // BOTTOM layer – the card background
7     Container(
8       width:300, height:200,
9       decoration: BoxDecoration(
10        gradient: const LinearGradient(
11          colors:[Color(0xFF5C4FE8), Color(0xFF9C27B0)],
12        borderRadius: BorderRadius.circular(20),
13      ),
14    ),
15
16    // MIDDLE layer – positioned exactly where you want
17    Positioned(
18      top:12, left:12,
19      child: Container(
20        padding: const EdgeInsets.symmetric(horizontal:12, vertical:6),
21        decoration: BoxDecoration(
22          color: Colors.red, borderRadius: BorderRadius.circular(20)),
23        child: const Text("SALE", style:TextStyle(color:Colors.white, fontSize:12)),
24      ),
25    ),
26
27    // TOP layer – bottom overlay with gradient
28    Positioned(
29      bottom:0, left:0, right:0,
30      child: Container(
31        padding: const EdgeInsets.all(16),
32        decoration: BoxDecoration(
33          borderRadius: const BorderRadius.only(
34            bottomLeft:Radius.circular(20), bottomRight:Radius.circular(20)),
35          gradient: LinearGradient(
36            begin:Alignment.bottomCenter, end:Alignment.topCenter,
37            colors:[Colors.black87, Colors.transparent]),
38        ),
```


Need to...	Widget to Use
Tags/chips wrap to next line	Wrap
Avoid notch/status bar	SafeArea
Center a widget	Center
Maintain aspect ratio	AspectRatio(aspectRatio: 16/9)
Dynamic list of widgets	ListView.builder
Two-column grid	GridView.builder

Lists, GridView & Scrolling

8.1 ListView.builder — The Most Important List Widget

```

1 // Basic ListView.builder
2 class ProductListScreen extends StatelessWidget {
3   final List<Map<String,dynamic>> products;
4   const ProductListScreen({super.key, required this.products});
5
6   @override
7   Widget build(BuildContext context) {
8     return Scaffold(
9       appBar: AppBar(title: const Text("Products")),
10      body: ListView.builder(
11        padding: const EdgeInsets.all(16),
12        itemCount: products.length,
13        itemBuilder: (context, index) {
14          final product = products[index];
15          return Card(
16            margin: const EdgeInsets.only(bottom: 12),
17            elevation: 2,
18            child: ListTile(
19              contentPadding: const EdgeInsets.all(16),
20              leading: CircleAvatar(
21                backgroundColor: const Color(0xFF5C4FE8),
22                child: Text("${index + 1}",
23                  style: const TextStyle(color: Colors.white, fontWeight: FontWeight.bold)),
24              ),
25              title: Text(product["name"],
26                style: const TextStyle(fontWeight: FontWeight.bold)),
27              subtitle: Text("\$${product['price']}",
28                style: const TextStyle(color: Color(0xFF5C4FE8), fontWeight: FontWeight.bold))

```

[illegible]

8.2 GridView.builder

```
1 GridView.builder(
2   shrinkWrap: true, // use inside a Column – takes only needed height
3   physics: const NeverScrollableScrollPhysics(), // disable own scroll
4   gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
5     crossAxisCount: 2, // 2 columns
6     crossAxisSpacing: 12, // horizontal gap between cards
7     mainAxisSpacing: 12, // vertical gap between cards
8     childAspectRatio: 0.75, // card height = card width / 0.75
9   ),
10  itemCount: products.length,
11  itemBuilder: (ctx, i) {
12    return ProductCard(product: products[i]);
13  },
14 )
```

8.3 Pull-to-Refresh

```
1 // Wrap your ListView in RefreshIndicator for pull-to-refresh
2 RefreshIndicator(
3   color: const Color(0xFF5C4FE8),
4   onRefresh: () async {
5     // This runs when user pulls down
6     await fetchFreshData(); // your API call
7     setState(() {}); // rebuild with new data
8   },
9   child: ListView.builder(
10    itemCount: items.length,
11    itemBuilder: (ctx, i) => ItemCard(item: items[i]),
12  ),
13 )
```


Navigation — Screens & Routes

9.1 Navigator — Push and Pop

65

```
33 (route) => false, // false = remove everything
34 );
```

9.2 Bottom Navigation with IndexedStack

```
1 class MainScreen extends StatefulWidget {
2   const MainScreen({super.key});
3   @override State<MainScreen> createState() => _MainScreenState();
4 }
5
6 class _MainScreenState extends State<MainScreen> {
7   int _current = 0;
8
9   // The 4 screens – IndexedStack keeps them alive when you switch
10  final _screens = const [
11    HomeScreen(),
12    SearchScreen(),
13    CartScreen(),
14    ProfileScreen(),
15  ];
16
17  @override
18  Widget build(BuildContext context) {
19    return Scaffold(
20      // IndexedStack shows only current screen but keeps all alive
21      body: IndexedStack(
22        index: _current,
23        children: _screens,
24      ),
25      bottomNavigationBar: NavigationBar(
26        selectedIndex: _current,
27        onDestinationSelected: (i) => setState(() => _current = i),
28        destinations: const [
29          NavigationDestination(
30            icon: Icon(Icons.home_outlined), selectedIcon: Icon(Icons.home),
31            label: "Home"),
32          NavigationDestination(
33            icon: Icon(Icons.search_outlined), selectedIcon: Icon(Icons.search),
34            label: "Search"),
35          NavigationDestination(
36            icon: Icon(Icons.shopping_bag_outlined),
37            selectedIcon: Icon(Icons.shopping_bag), label: "Cart"),
38          NavigationDestination(
39            icon: Icon(Icons.person_outline), selectedIcon: Icon(Icons.person),
```

```
40   label: "Profile"),  
41   ],  
42   ),  
43   );  
44   }  
45   }
```

State Management — Provider & Riverpod

10.1 The Problem

State Management makes data available to the ENTIRE app.
Any widget that needs the cart data rebuilds automatically.

10.2 Provider — Step by Step

68

```

16 void addItem(Map<String, dynamic> product) {
17   _items.add(product);
18   notifyListeners(); // tells ALL listening widgets to rebuild
19 }
20
21
22 void removeItem(int index) {
23   _items.removeAt(index);
24   notifyListeners();
25 }
26
27 void clear() {
28   _items.clear();
29   notifyListeners();
30 }
31 }
32
33 // ■■ Step 3: Wrap app in ChangeNotifierProvider ██████████
34 void main() {
35   runApp(
36     ChangeNotifierProvider(
37       create: (_) => CartProvider(),
38       child: const MyApp(),
39     ),
40   );
41 }
42
43 // ■■ Step 4: Read data – LISTEN (rebuilds when changed) ████████
44 class CartBadge extends StatelessWidget {
45   const CartBadge({super.key});
46   @override
47   Widget build(BuildContext context) {
48     final cart = context.watch<CartProvider>(); // listen
49     return Badge(
50       label: Text("${cart.count}"),
51       child: const Icon(Icons.shopping_cart),
52     );
53   }
54 }
55
56 // ■■ Step 5: Trigger changes – READ (no rebuild here) ██████████
57 class AddButton extends StatelessWidget {
58   final Map<String, dynamic> product;
59   const AddButton({super.key, required this.product});
60   @override

```

```
61 Widget build(BuildContext context) {
62   return ElevatedButton(
63     onPressed: () {
64       // context.read = no rebuild, just trigger action
65       context.read<CartProvider>().addItem(product);
66       ScaffoldMessenger.of(context).showSnackBar(
67         SnackBar(content: Text("${product['name']} added!")),
68       );
69     },
70     child: const Text("Add to Cart"),
71   );
72 }
73 }
```

REST APIs & Local Storage

11.1 HTTP Basics

11.2 Making API Calls with Dio

71


```
19
20 // After login – save token securely
21 await storage.write(key: "authToken", value: "Bearer eyJhbGc...");
22
23 // On each API call – read the token
24 final token = await storage.read(key: "authToken");
25
26 // On logout – delete the token
27 await storage.delete(key: "authToken");
```

CHAPTER 12

Firestore — Auth, Firestore & Storage

Build a complete backend without writing a single server line

12.1 Firebase Services

Service	What It Gives You	Use In App
Authentication	Email, Google, Phone login	Every app that has accounts
Firestore	Real-time cloud database	Chat, social feed, orders
Storage	File/image storage in cloud	Profile photos, uploads
FCM	Push notifications to devices	Alerts, order updates
Analytics	Track user behaviour	See what features work
Crashlytics	Automatic crash reports	Fix production bugs

12.2 Firebase Authentication

```
1 import 'package:firebase_auth/firebase_auth.dart';
2
3 class AuthService {
4   final _auth = FirebaseAuth.instance;
5
6   // Get current logged-in user (null if not logged in)
7   User? get currentUser => _auth.currentUser;
8
9   // Stream that fires every time login state changes
10  Stream<User?> get authChanges => _auth.authStateChanges();
11
12  // Register with email and password
13  Future<String?> register(String email, String password) async {
14    try {
15      final cred = await _auth.createUserWithEmailAndPassword(
```

```

16 email: email, password: password);
17 await cred.user?.sendEmailVerification();
18 return null; // null = success
19 } on FirebaseAuthException catch (e) {
20   return _mapError(e.code); // return error message
21 }
22 }
23
24 // Login with email and password
25 Future<String?> login(String email, String password) async {
26   try {
27     await _auth.signInWithEmailAndPassword(email: email, password: password);
28     return null; // null = success
29   } on FirebaseAuthException catch (e) {
30     return _mapError(e.code);
31   }
32 }
33
34 // Logout
35 Future<void> logout() => _auth.signOut();
36
37 // Password reset email
38 Future<void> resetPassword(String email) =>
39   _auth.sendPasswordResetEmail(email: email);
40
41 String _mapError(String code) {
42   switch (code) {
43     case 'email-already-in-use': return "Email already registered";
44     case 'wrong-password': return "Incorrect password";
45     case 'user-not-found': return "No account found";
46     case 'weak-password': return "Password too weak (min 6 chars)";
47     default: return "Error: $code";
48   }
49 }
50 }
51
52 // ■■ Auth Gate – shows right screen based on login status ■■■■■■
53 class AuthGate extends StatelessWidget {
54   const AuthGate({super.key});
55   @override
56   Widget build(BuildContext context) {
57     return StreamBuilder<User?>(
58       stream: FirebaseAuth.instance.authStateChanges(),
59       builder: (ctx, snapshot) {
60         if (snapshot.connectionState == ConnectionState.waiting)

```

```
61 return const SplashScreen();
62 return snapshot.hasData ? const HomeScreen() : const LoginScreen();
63 },
64 );
65 }
66 }
```

12.3 Firestore — Real-time Database

[illegible]

[illegible]

CHAPTER 13

Architecture & Clean Code

Write code that other developers (including future you) can read and maintain

13.1 Clean Architecture — 3 Layers

Layer	Responsibility	Example
Presentation	UI only — widgets, screens, state providers	HomeScreen, ProductCard, CartProvider
Domain	Business rules — pure Dart, no Flutter/Firebase	GetProductsUseCase, AddToCartUseCase
Data	Fetch and store data — repositories, models, sources	ProductRepository, ApiSource, FirestoreSource

13.2 Recommended Folder Structure

```
1 lib/
2 core/
3   constants.dart // app-wide constants (colors, URLs)
4   theme.dart // ThemeData for the whole app
5   utils/ // helper functions
6   errors/ // custom exceptions
7
8   features/
9     auth/
10    data/
11    auth_repository.dart
12    auth_remote_source.dart
13    user_model.dart
14    domain/
15    user_entity.dart
16    auth_usecase.dart
17    presentation/
18    login_screen.dart
```

```
19 register_screen.dart
20 auth_provider.dart
21
22 products/
23 data/ ...
24 domain/ ...
25 presentation/ ...
26
27 cart/
28 data/ ...
29 domain/ ...
30 presentation/ ...
31
32 shared/
33 widgets/
34 app_button.dart // your custom button used everywhere
35 app_text_field.dart // your custom text input
36 loading_widget.dart
37 error_widget.dart
38 services/
39 analytics_service.dart
40
41 main.dart
```

13.3 Reusable Widget Pattern

```
1 // Build your own component library – use across the entire app
2 class AppButton extends StatelessWidget {
3   final String label;
4   final VoidCallback onPressed;
5   final bool isLoading;
6   final bool outlined;
7   final Color? color;
8
9   const AppButton({
10     super.key,
11     required this.label,
12     required this.onPressed,
13     this.isLoading = false,
14     this.outlined = false,
15     this.color,
16   });
17
18   @override
```



```
19 Widget build(BuildContext context) {
20   final col = color ?? const Color(0xFF5C4FE8);
21   final child = isLoading
22     ? SizedBox(width:20, height:20,
23       child: CircularProgressIndicator(color:Colors.white, strokeWidth:2))
24     : Text(label, style: const TextStyle(fontSize:15, fontWeight:FontWeight.bold));
25
26   return SizedBox(
27     width: double.infinity,
28     height: 52,
29     child: outlined
30       ? OutlinedButton(
31         onPressed: isLoading ? null : onPressed,
32         style: OutlinedButton.styleFrom(
33           side: BorderSide(color:col),
34           shape: RoundedRectangleBorder(borderRadius:BorderRadius.circular(14)),
35         ),
36         child: DefaultTextStyle(
37           style: TextStyle(color:col, fontWeight:FontWeight.bold),
38           child: child),
39       )
40       : ElevatedButton(
41         onPressed: isLoading ? null : onPressed,
42         style: ElevatedButton.styleFrom(
43           backgroundColor: col,
44           shape: RoundedRectangleBorder(borderRadius:BorderRadius.circular(14)),
45         ),
46         child: child,
47       ),
48   );
49 }
50 }
51
52 // Usage everywhere in the app – consistent, no duplicate code:
53 // AppButton(label: "Sign In", onPressed: _login)
54 // AppButton(label: "Cancel", onPressed: _cancel, outlined: true)
55 // AppButton(label: "Loading", onPressed: _save, isLoading: _isSaving)
```

Testing, Optimization & Deployment

14.1 Types of Tests

Type	Tests What	Tool	When
Unit Test	Functions & classes in isolation	test	Always — for all business logic
Widget Test	Individual widgets render correctly	flutter_test	Always — for important screens
Integration Test	Full app flows end-to-end	integration_test	Before major releases

82


```
6 # storePassword=yourpassword
7 # keyPassword=yourpassword
8 # keyAlias=my-key
9 # storeFile=/path/to/my-release-key.jks
10
11 # ■■ Step 3: Build release App Bundle ████████████████████
12 flutter build appbundle --release --obfuscate --split-debug-info=build/debug
13 # File: build/app/outputs/bundle/release/app-release.aab
14
15 # ■■ Step 4: Upload to Google Play Console ████████████████████
16 # Go to play.google.com/console
17 # Create your app → Production → Create new release → Upload .aab
```

14.4 Play Store Checklist

Item	Requirement
App Icon	512×512 PNG, no transparency, no rounded corners
Screenshots	Minimum 2 phone screenshots required
Privacy Policy	Required for ALL apps — host free on sites.google.com
Data Safety	Declare what user data your app collects and why
Description	Up to 4000 characters — include important keywords
Content Rating	Complete IARC questionnaire honestly
Target SDK	Must meet Google's minimum required API level

✓ You Are Now a Flutter Developer ■

You have completed the full journey from Dart basics to production deployment.

What you can now build independently:

- ✓ Complete multi-screen apps with professional navigation
- ✓ Apps that fetch and display real data from REST APIs
- ✓ Apps with Firebase Auth, live database, and file storage
- ✓ Apps with proper state management (Provider / Riverpod)
- ✓ Apps following clean architecture principles
- ✓ Apps published on Google Play Store

Your next steps:

1. Build 3 real portfolio apps (chat app, e-commerce, your own idea)
2. Publish at least 1 app to the Play Store (even with 0 downloads)
3. Clean GitHub repos with README and demo screenshots
4. Apply for Flutter developer jobs or start freelancing on Fiverr/Upwork